# BioHORS

## Team instruction packet

FTC 2026-27 BIOBUZZ - auto, tuning, and every value left to measure

**What this is.** Everything the team needs to take the simulated auto to the real robot: how to rebuild the setup, how to tune Pedro Pathing v3, how to move a pose or change an action, and a list of every number that is still a placeholder.

| If you want to... | Go to |
|---|---|
| Get the code running and see the auto in AdvantageScope | Section 2 |
| Tune the drive so paths are followed on the real robot | Sections 3 and 4 |
| Make the shooters hold speed and the pollen score | Section 5 |
| Move where the robot drives to | Section 6 |
| Change a speed, wait, rpm or servo position | Section 7 |
| Know what is still a guess | Section 8 |

> **NOTE** Blue underlined text is a link. File names link to the file on GitHub (main branch). Use the PDF bookmarks panel, or the **Contents** link in every page footer, to jump around.

# Contents

1 **Where the auto stands**

What runs today, what it scores, and what is still made up

2 **Set up and replicate**

Clone, build, run the sim, open the logs in AdvantageScope

3 **Check the wiring first**

Ten minutes before any tuning: names, directions, gear ratio

4 **Tune Pedro Pathing v3**

AutoTune web page: Mecanum, Pinpoint, Foresight, Tests

5 **Tune the mechanisms**

Shooters, gates, flower bar, intake pollen count, aim

6 **Change a pose**

Move a waypoint: Zenith, AutoPoses, the tests

7 **Change an action value**

Speeds, waits, rpm, servo positions: which file, which line

8 **Every value to measure on the robot**

The full list, grouped, with where each one lives

9 **Attached files and logs**

What is in the zip and how to use each part

10 **Links and documentation**

Every page this packet points to

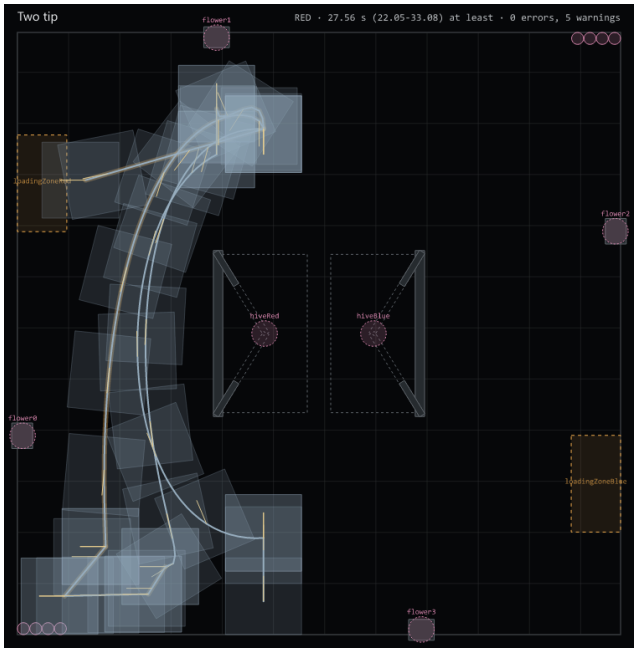
# 1 Where the auto stands

What runs today, what it scores, and what is still made up

Two autos run the same route, and both pass every sim test on both alliances:

| OpMode name | What it is | File |
|---|---|---|
| **Two tip** (group Generated) | Zenith auto: the route lives in `autos/two-tip.auto.json`, run by the SolversLib command runner. | `TeamCode/src/main/java/org/firstinspires/ftc/teamcode/zenith/BioHorsZenithRobot.java` |
| **Two Tip (state machine)** (group BioHORS) | Hand-written: Pedro paths and one explicit state machine per mechanism. Easier to read and change by hand. | `TeamCode/src/main/java/org/firstinspires/ftc/teamcode/auto/TwoTipStateMachineAuto.java` |
| **MainHORS TeleOp** | Driver control; the autos pre-select it. | `TeamCode/src/main/java/org/firstinspires/ftc/teamcode/teleop/MainHorsTeleOp.java` |

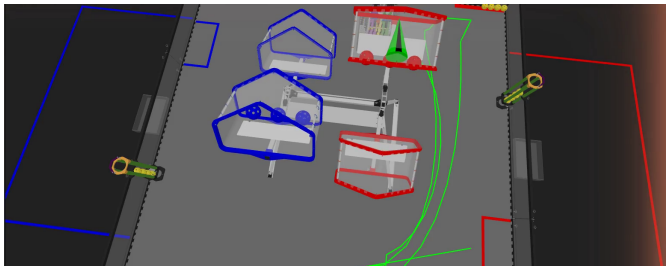
**The route (RED; BLUE is the point mirror).** Start on the south wall with 4 pollen, score them from *scoreSouth*, drive north and take pollen from the flower with the bar down, score from *scoreNorth*, come back down and sweep the garden row along the south wall with the bar stowed, drive back round the hive and score the third volley from *scoreNorth*, then park. In the sim it scores **12 of 12** pollen and tips the hive **twice**, touches nothing, and finishes in about **27.6 s** of the 30.



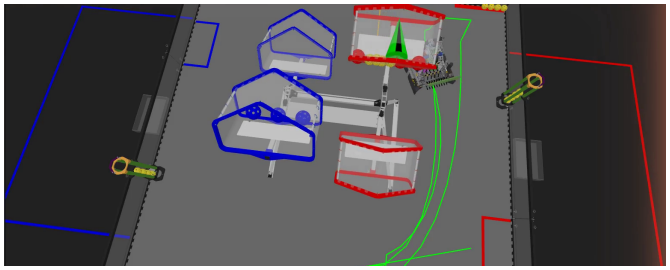

*Zenith render of the RED route (zenith render two-tip). Green dots are waypoints; the numbers are the steps.*

> **CAREFUL** **All of this is simulated.** No number on this robot is tuned yet. The poses come from FIRST's field CAD and the robot CAD, not a real field. The shot (exit angle and speed) is a what-if; the scoring spots and the auto rpm (1750) were picked in the sim to make that what-if shot land. **Filming one real shot is the most important measurement left** (Section 5d).

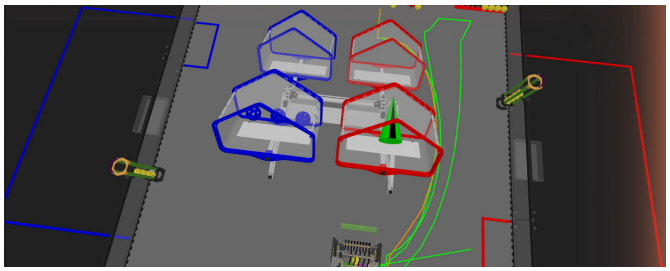
What it looks like in AdvantageScope (frames from the physics run; the robot is the CAD model, the green line is the planned route, the orange line the driven path):



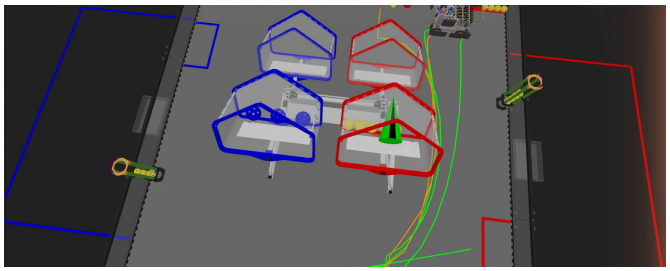
*1.5 s into the auto.*



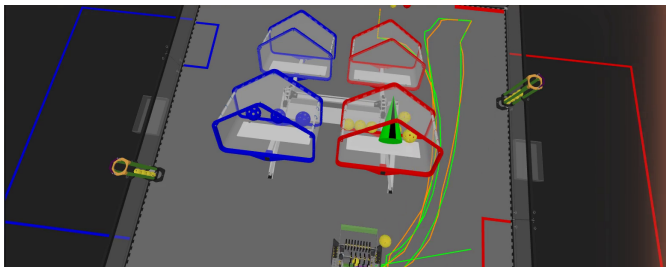
*4.6 s.*



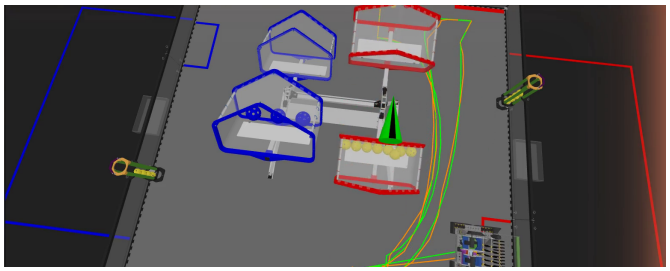
*9.5 s.*



*14.5 s.*



*20.5 s.*



*23.5 s.*

These frames were recorded just before the last small change (the garden sweep now goes 3.5 in further toward the corner and the flower pickup 0.35 in closer), so the robot in them stops a touch short of where it does now.

# 2 Set up and replicate

Clone, build, run the sim, open the logs in AdvantageScope

**What you need.** Android Studio (any recent one; its bundled JDK 17+ is what Gradle uses), git, and AdvantageScope 26 or newer to look at logs. Node.js 18+ only if you edit the Zenith auto.

**1. Get the code.**

```
git clone https://github.com/ZOGWORK/BioHORS.git
cd BioHORS
```

**2. Run every test (the simulator).** No robot needed; it takes about a minute. It runs both autos as full matches on both alliances, with and without ball physics, and writes logs to `TeamCode/build/sim/`.

```
# Windows PowerShell or cmd
.\gradlew.bat :TeamCode:testDebugUnitTest

# macOS / Linux / Git Bash
./gradlew :TeamCode:testDebugUnitTest
```

> **TIP** If Gradle says JAVA_HOME is not set, point it at Android Studio's JDK, e.g. `$env:JAVA_HOME = "C:\Program Files\Android\Android Studio\jbr"` in PowerShell.

**3. Put it on the robot.** Open the folder in Android Studio, plug in the Control Hub (or join its Wi-Fi), pick the **TeamCode** run configuration and press Run. On the Driver Station the OpModes appear under the names in Section 1.

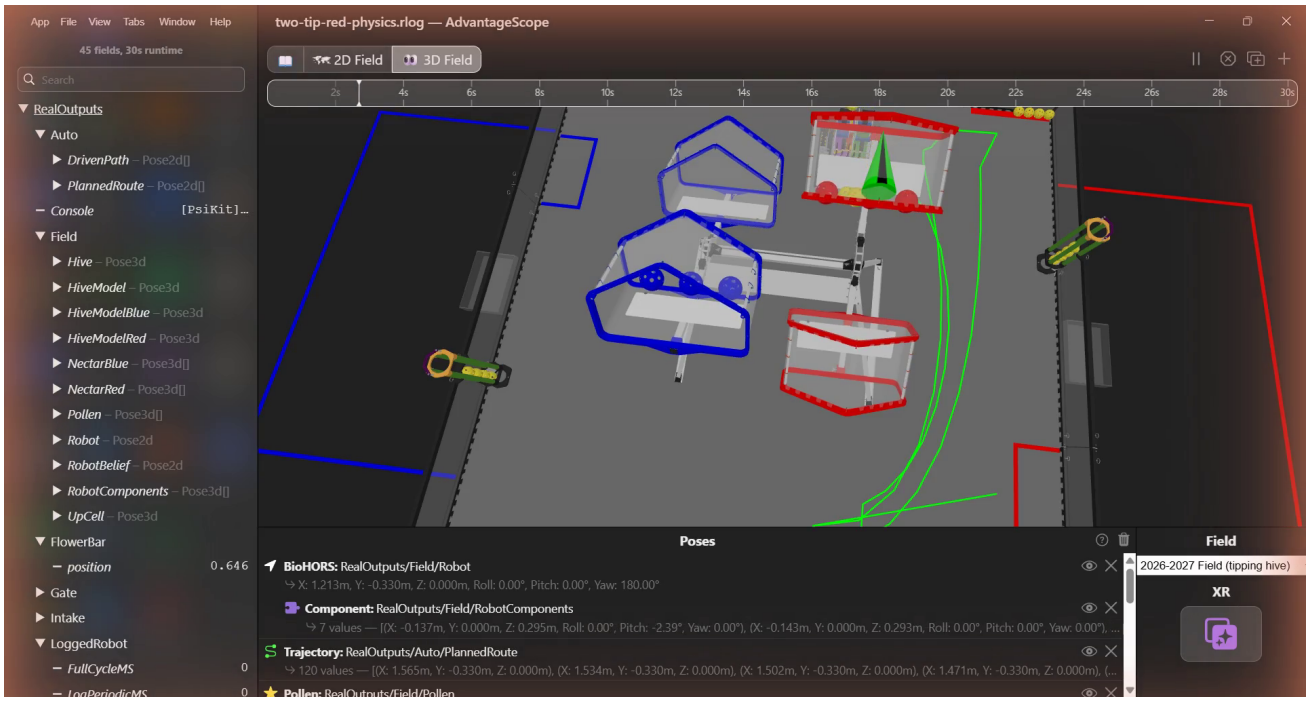
**4. Install the AdvantageScope assets** (once per laptop). Open AdvantageScope once and pick the 2026-2027 FTC field so it downloads the field model, close it, then run (Python with numpy):

```
python advantagescope/tools/build_hive_assets.py
```

That writes *2026-2027 Field (tipping hive)*, *Hive (Red)* and *Hive (Blue)* into `%APPDATA%/AdvantageScope/userAssets`. Copy the folder `advantagescope/Robot_BioHORS` (the robot CAD model) into the same folder by hand, then restart AdvantageScope.

**5. Open a log.** Drag any `.rlog` from `TeamCode/build/sim/` (or from the attached zip) onto AdvantageScope. Add a **3D Field** tab, pick the field *2026-2027 Field (tipping hive)*, and drag these keys in:

| Drag this key | As | Shows |
|---|---|---|
| RealOutputs/Field/Robot | Robot -> BioHORS | the robot (CAD model) |
| RealOutputs/Field/RobotComponents | Component (under the robot) | intake rollers spinning, flower bar moving |
| RealOutputs/Auto/PlannedRoute | Trajectory | the planned path (green) |
| RealOutputs/Auto/DrivenPath | Trajectory | where the robot really went |
| RealOutputs/Field/Pollen, NectarRed, NectarBlue | Game piece | every ball, flying and landing |
| RealOutputs/Field/HiveModelRed / Blue | Hive (Red) / Hive (Blue) | the hives, tipping |




*AdvantageScope with two-tip-red-physics.rlog: the keys on the left, the 3D field, and the Poses list at the bottom.*

# 3 Check the wiring first

Ten minutes before any tuning: names, directions, gear ratio

Do this before any tuning. A motor spinning the wrong way makes every tuner give nonsense. The full checklist is section 0 of `docs/TUNING.md`.

| Check | How | If wrong, change |
|---|---|---|
| Robot Controller config names | Must match exactly: frontLeft, backLeft, frontRight, backRight, intakeLeft, intakeRight, shooterRight (pollen), shooterLeft (nectar), gateRight (pollen), gateLeft (nectar), flowerLeft, flowerRight, pinpoint | The RC configuration on the Driver Station |
| Which shooter is pollen | Spin pollen alone in the Shooter Tuner; see which wheel turns. (Pollen on the right is assumed, not confirmed.) | `MechanismConstants` hardware names |
| Shooter directions | Both must throw the ball OUT at positive power | `POLLEN_SHOOTER_DIRECTION`, `NECTAR_SHOOTER_DIRECTION` |
| Shooter gear ratio | 1:1 is confirmed; if it ever changes, ticksPerRev changes | `MechanismConstants.ticksPerRev` |
| Drive directions | Mecanum Tuner (Section 4, step 1) | `pedro/Constants` drivetrain |
| Flower bar servos | They face away from each other; the right one is reversed in code. Both must move the bar the same way | `FlowerBarController` |
| Intake | Both rollers pull IN at positive power | `IntakeController` |

# 4 Tune Pedro Pathing v3

AutoTune web page: Mecanum, Pinpoint, Foresight, Tests

Pedro Pathing v3 tunes on the robot through **AutoTune**, a web page the robot itself serves. You do not run separate tuning OpModes any more: the procedures are listed on the page. This robot's procedures are in `TeamCode/src/main/java/org/firstinspires/ftc/teamcode/pedro/Tuning.java`; every result goes into `TeamCode/src/main/java/org/firstinspires/ftc/teamcode/pedro/Constants.java`. Official guide: https://pedropathing.com/docs/pathing/tuning.

Pedro pages to keep open while tuning: Mecanum, Pinpoint, Foresight, Tests, Constants, and what changed in Pedro 3.

### Getting to the page

| Step | Do |
|---|---|
| 1 | Deploy the code (Section 2, step 3). Fresh battery; a clear space of about 8 x 8 ft (Foresight drives fast in straight lines). |
| 2 | Connect the laptop to the Robot Controller's Wi-Fi (the same network the Driver Station uses). |
| 3 | Open http://192.168.43.1:10158 in a browser. That is the AutoTune page. |
| 4 | Pick a procedure from the list, follow its on-screen prompts, and run it with the Driver Station when it asks. |
| 5 | When it finishes, open its **Java** tab and copy the lines it gives you into `pedro/Constants.java` (the place for each is below). Redeploy before the next procedure. |

## The four procedures, in order

| # | Procedure | What it finds | Paste into | Done when |
|---|---|---|---|---|
| 1 | Mecanum Tuner | Which way each drive motor must spin | `drivetrainConfig` motor directions | Every wheel drives the robot forward at positive power |
| 2 | Pinpoint Tuner | Pod directions and pod offsets (4-bar pods, device "pinpoint") | `localizerConfig`: xPodOffset (now -2.84, from CAD: compare), yPodOffset (0, placeholder), x/yPodDirection | Push forward: x goes up. Push left: y goes up. Spin 10 turns in place: drift under 1 in |
| 3 | Foresight Tuner | Top speeds, natural deceleration, heading P, coast/brake, brake coefficients | `foresightConfig()` and the MAX_FORWARD / MAX_STRAFE / *_DECEL constants | It finishes and prints values |
| 4 | Tests | Follows test paths with the tuned follower; includes the localization test | nothing; it checks | A 48 in line ends within 0.5 in; a 90 deg turn settles within 2 deg |

> **CAREFUL** **Keep Zenith in step.** After Foresight, copy the same top speeds and decelerations into `autos/robot.json` (*kinematics*: maxForwardVelInPerS, maxStrafeVelInPerS, forwardDecelInPerS2, strafeDecelInPerS2) and change their provenance to CALIBRATED FROM ROBOT. Zenith's time estimate uses them, and the sim drive model reads them too.

> **TIP** Also copy the Pinpoint pod offsets into the sim, `X/Y_POD_OFFSET_IN` in `TeamCode/src/test/.../sim/BioHorsSim.java`, so the simulator matches the robot.

**Then, once the robot drives real paths:** record a few auto runs, and refit Zenith's acceleration and settle time from the *robot* traces (not the sim ones) with `npx zenith calibrate --write`.

# 5 Tune the mechanisms

Shooters, gates, flower bar, intake pollen count, aim

Each step has a full write-up in `docs/TUNING.md` (the section number is given). All values live in `TeamCode/src/main/java/org/firstinspires/ftc/teamcode/subsystems/MechanismConstants.java` unless the table says otherwise.

## 5a. Shooters (TUNING.md section 3)

Run the **Shooter Tuner** OpMode (group *Tuning*), robot on blocks, no pollen, fresh battery. It tunes pollen, then nectar, then checks both together, and prints lines. Paste them into each shooter's "tuned values" block in MechanismConstants. It sets kS, the feedforward reference and kP; it does **not** pick the target rpm.

## 5b. Target rpm (field test)

Teleop pollen target is 1600 rpm (the team's test value); the auto uses `AUTO_POLLEN_RPM` = 1750 (picked in the sim). Shoot from the auto's spots and trim in the teleop: **dpad left/right** trims pollen, **X / A** trims nectar, 50 rpm a press. Write the rpm that lands 10 of 10 back into the file.

## 5c. Gates, flower bar, intake (sections 4, 4b, 4c)

| What | How to find it | Values |
|---|---|---|
| Gates | Move each gate servo by hand in small steps (e.g. with a servo tester or a short test OpMode); write down where it fully blocks and fully clears a pollen. | POLLEN/NECTAR_GATE_OPEN 0.67, CLOSED 0.485 |
| Flower bar | STOWED: folded back so nothing sticks past the frame (the auto drives 0.13 in off the wall!). DOWN: bar tip seats in the flower. SHOOTING: clear of the shot. | FLOWER_BAR_STOWED 0.05, DOWN 0.80, SHOOTING 0.65 |
| Intake pollen count | Run the intake empty, read the speed (free); load the hopper, read it again (loaded). The auto counts pollen from how much the intake slows. | INTAKE_FREE_TICKS_PER_S 2800, INTAKE_LOADED_TICKS_PER_S 2100, settle 0.4 s, filter 0.3 |

## 5d. The shot (section 9a): the most important measurement left

Film one pollen shot from the side at 1750 rpm, with a ruler in frame. Measure the exit angle and the ball speed. Put them in the sim's ShotModel (`TeamCode/src/test/.../sim/`), rerun `ShotSweepTest`, and it reports the rpm and distance that seat a pollen. Only then move the scoring spots (Section 6) and set AUTO_POLLEN_RPM.

> **CAREFUL** With the CAD exit angle (44 deg) no spot inside the field scores at any rpm in the sim. The auto scores today only because of the what-if 70 deg shot. Do not trust the scoring spots until a shot is filmed.

## 5e. Aim (section 8)

In `TeamCode/src/main/java/org/firstinspires/ftc/teamcode/aim/AimController.java`: confirm the shooter fires out of the robot's front (SHOOTER_FACING_OFFSET_RAD = 0), then tune kP (1.2) until the robot turns onto the hive without oscillating; MIN_YAW (0.06) is the smallest command that still moves it.

# 6 Change a pose

Move a waypoint: Zenith, AutoPoses, the tests

A **pose** is a point and a heading on the field: x and y in inches, heading in radians. The auto drives between named poses called **waypoints**.

## The field frame (used everywhere: auto, aim, sim)

| | |
|---|---|
| Origin | Centre of the field |
| +X | Toward the audience's right (the BLUE side) |
| +Y | Away from the audience |
| Heading | Radians, counter-clockwise from +X. 90 deg = pi/2 = 1.5708 faces +Y (north); 180 deg = pi = 3.1416 faces -X (west); -90 deg faces south |
| Units | Inches. The walls are at x, y = +/-70.674 (FIRST CAD) |
| Alliances | Write RED only. BLUE is the point mirror: (x, y, h) -> (-x, -y, h + pi), done in code |

## The waypoints today (RED)

| Waypoint | x in | y in | heading | What it is |
|---|---|---|---|---|
| start | -13 | -62.87 | 90 deg | Back to the south wall, facing the hive |
| scoreSouth | -13 | -48 | 90 deg | First volley |
| scoreNorth | -13 | 48 | -90 deg | Second and third volley |
| flowerNorthApproach | -24 | 42 | 90 deg | Lines up below the flower |
| flowerNorthPickup | -24 | 52.75 | 90 deg | Bar down, its tip at the flower |
| aboveGardenLane | -36 | -54.7 | 180 deg | Above the garden row |
| gardenSweepStart | -40 | -61.17 | 180 deg | Start of the sweep, by the south wall |
| gardenSweepEnd | -59.6 | -61.62 | 180 deg | Front roller over the corner pollen |
| offGardenLane | -50 | -50 | 180 deg | Back off the wall after the sweep |
| park | -54.7 | 36 | 180 deg | End of the auto |

## How to move one

The two autos keep their own copy of the points. Change both; a test fails if they disagree.

| Step | Do this | Where |
|---|---|---|
| 1 | Edit the waypoint's xIn / yIn / headingRad, and update its *provenance* text (say what you measured and when). Easiest in the Zenith desktop app: `npx zenith open`, drag the point, save. Or edit the JSON by hand. | `autos/waypoints.json` |
| 2 | Check it: no wall, hive or flower in the way, and it fits in 30 s. | `npx zenith validate`<br>`npx zenith estimate autos/two-tip.auto.json` |
| 3 | Copy it to the robot and refresh the generated Java. | `npx zenith deploy`<br>`npx zenith codegen autos/two-tip.auto.json` |
| 4 | Copy the same number into the state-machine auto's pose (named in CAPITALS, e.g. `GARDEN_SWEEP_END`). | `TeamCode/src/main/java/org/firstinspires/ftc/teamcode/auto/AutoPoses.java` |
| 5 | Run the tests. AutoPosesTest checks the two copies are equal; the match tests check both autos still score, tip, avoid contact and finish on time on both alliances. | `.\gradlew.bat :TeamCode:testDebugUnitTest` |
| 6 | Open `TeamCode/build/sim/two-tip-red.rlog` in AdvantageScope and watch it. | Section 2 |
| 7 | Branch, commit, open a PR (never push to main). | GitHub |

> **NOTE** **Curved legs.** Some legs are Bezier curves whose heading is split into short linear pieces (Pedro 3.0.2 cannot follow a tangent heading inside a piecewise heading without spinning). If you move a waypoint at the end of a curve, the heading pieces in `two-tip.auto.json` and `AutoPoses` were generated by `tools/zenith_headings.py`; regenerate them the same way, or the robot may twist on that leg.

> **TIP** **Measuring a waypoint on the real field:** put the robot where it should be, run the teleop, and read the Pinpoint pose off the telemetry (after it was started at the real start pose). That number, in the field frame, is the waypoint.

# 7 Change an action value

Speeds, waits, rpm, servo positions: which file, which line

An **action value** is anything the robot does rather than where it goes: how fast it drives a leg, how long it waits, the shooter rpm, a servo position. Each one is a named constant near the top of its file.

| To change... | Edit | File |
|---|---|---|
| Shooter rpm in auto | `AUTO_POLLEN_RPM` (1750) | `TeamCode/src/main/java/org/firstinspires/ftc/teamcode/subsystems/MechanismConstants.java` |
| Shooter rpm in teleop | `POLLEN_SHOOTER.targetRpm` / nectar (1600) | same |
| Gate / flower bar positions | `*_GATE_OPEN/CLOSED`, `FLOWER_BAR_*` | same |
| Drive speed on a leg (state machine) | `DEFAULT_SPEED` 0.8, `PICKUP_SPEED` 0.3, `GARDEN_SWEEP_SPEED` 0.2, `SLIDE_SPEED` 0.3, `OFF_LANE_SPEED` 0.6 | `TeamCode/src/main/java/org/firstinspires/ftc/teamcode/auto/TwoTipStateMachineAuto.java` |
| Waits (state machine) | `FLOWER_WAIT_S` 1.0, `GARDEN_WAIT_S` 0.8, `SPIN_UP_AFTER_S` 0.5, `SWEEP_TIMEOUT_S` 3.0 | `TeamCode/src/main/java/org/firstinspires/ftc/teamcode/auto/TwoTipStateMachineAuto.java` |
| Drive speed / waits (Zenith auto) | a path step's *speedFraction*, a wait step's *timeoutS* / *seconds* (or in the Zenith app); then `zenith validate`, `deploy`, `codegen` | `autos/two-tip.auto.json` |
| Volley timing (both autos) | `SPINUP_TIMEOUT_S` 2.0, `FEED_S_PER_PIECE` 0.4, `FEED_EXTRA_S` 0.2 | `TeamCode/src/main/java/org/firstinspires/ftc/teamcode/zenith/ScoreCommand.java` |
| Teleop stick feel | `STICK_DEADBAND`, `*_EXPO`, `MAX_*` | `TeamCode/src/main/java/org/firstinspires/ftc/teamcode/subsystems/DriveController.java` |
| Intake power | `INTAKE_POWER` 1.0, `OUTTAKE_POWER` -1.0 | `TeamCode/src/main/java/org/firstinspires/ftc/teamcode/subsystems/IntakeController.java` |
| Zenith's idea of the robot | speeds, accel, settle times, footprint, mouth | `autos/robot.json` |

## Every time you change a value

| | |
|---|---|
| 1. Label it | Every number has a provenance comment. Change PLACEHOLDER to MEASURED / CALIBRATED FROM ROBOT / SET BY HAND, and say how you got it. Never present a guess as measured. |
| 2. Test it | `.\gradlew.bat :TeamCode:testDebugUnitTest`: if a match test fails, the sim thinks the change breaks the auto. Read why before overriding. |
| 3. Ship it | Branch, commit, PR. Greptile reviews it; merge when it is clean. |

**CAREFUL** The sim tests use a few made-up plant numbers too (the shot, motor time constants). Do not change a PLACEHOLDER shot or sim constant just to make the auto score higher in the sim.

# 8 Every value to measure on the robot

The full list, grouped, with where each one lives

Everything below is a guess, a spec sheet number or a sim result. Work down the list roughly in this order: wiring, Pedro, shooters, mechanisms, then the field. The full generated list (every line with a PLACEHOLDER / NEEDS MEASUREMENT / SET FROM SIM label) is in the zip as `open-values.txt`.

| Group | Value (file) | Now | Label | How to measure |
|---|---|---|---|---|
| Field | All 10 waypoints (waypoints.json + AutoPoses) | Section 6 | SET BY HAND / SET FROM SIM | Drive there on a real field, read the Pinpoint |
| Field | Teleop start pose (MainHorsTeleOp DEFAULT_START_X/Y) | -13, -62.87 | NEEDS MEASUREMENT | Same as the auto start |
| Pinpoint | yPodOffset (pedro/Constants) | 0 | PLACEHOLDER | Pinpoint Tuner |
| Pinpoint | xPodOffset | -2.84 | MEASURED FROM CAD | Pinpoint Tuner: compare |
| Pinpoint | x/y pod directions | FORWARD | PLACEHOLDER | Pinpoint Tuner |
| Drive | MAX_FORWARD / MAX_STRAFE in/s | 60 / 50 | SET FROM SIM | Foresight Tuner |
| Drive | FORWARD / STRAFE_DECEL in/s^2 | 60 / 50 | SET FROM SIM | Foresight Tuner |
| Drive | coast / brake kV, heading P | 0.02 / 0.05, 3.0 | SET FROM SIM | Foresight Tuner |
| Drive | heading / linear / quadratic brake coefficients | example | PLACEHOLDER | Foresight Tuner |
| Drive | translational P (fwd, strafe) | 0.12/0.04, 0.15/0.06 | SET FROM SIM | Tests: 48 in line within 0.5 in |
| Zenith | robot.json speeds and decels | 60/50/60/50 | SET FROM SIM | Copy from Foresight |
| Zenith | robot.json accelInPerS2, settleS | 220, 0.4 | CALIBRATED FROM SIM | zenith calibrate on robot traces |
| Zenith | robot.json maxAngularVelRadPerS | 5 | PLACEHOLDER | Spin at full yaw, read the Pinpoint |
| Zenith | robot.json shooter settleS | 0.25 | PLACEHOLDER | Time from at-speed to first good shot |
| Shooter | kP, kI, kD, kF, kS (MechanismConstants) | 0.001, 0.0005, 0, 1, 0 | PLACEHOLDER | Shooter Tuner |
| Shooter | ffReferenceMaxTicksPerSec | 2800 | SPEC (no load) | Shooter Tuner |
| Shooter | integralLimit, derivative/rpm/power alphas | 50, 0.9, 0.72, 0.5 | PLACEHOLDER | Leave unless noisy |
| Shooter | rpmTolerance | 75 rpm | PLACEHOLDER | Shot-to-shot spread vs rpm |
| Shooter | Pollen / nectar targetRpm | 1600 / 1600 | SET BY HAND | Field test (5b) |
| Shooter | AUTO_POLLEN_RPM | 1750 | PLACEHOLDER, SET FROM SIM | After the shot is filmed (5d) |
| Shooter | Pollen motor = shooterRight | - | assumed | Section 3 |
| Shooter | Shooter directions | FORWARD | PLACEHOLDER | Section 3 |
| Gates | POLLEN/NECTAR_GATE_OPEN / CLOSED | 0.67 / 0.485 | PLACEHOLDER | Move by hand (5c) |
| Flower bar | FLOWER_BAR_STOWED / DOWN / SHOOTING | 0.05 / 0.80 / 0.65 | PLACEHOLDER | Move by hand (5c) |
| Intake | INTAKE_POWER / OUTTAKE_POWER | 1.0 / -1.0 | PLACEHOLDER | Collect test |
| Intake | INTAKE_FREE / LOADED_TICKS_PER_S | 2800 / 2100 | PLACEHOLDER | Read empty and full (5c) |

| Group | Value (file) | Now | Label | How to measure |
|---|---|---|---|---|
| Intake | INTAKE_SENSE_SETTLE_S, FILTER_ALPHA | 0.4, 0.3 | PLACEHOLDER | Watch the count on telemetry |
| Aim | SHOOTER_FACING_OFFSET_RAD | 0 | PLACEHOLDER | Which way the shot leaves |
| Aim | kP, MAX_YAW, MIN_YAW, tolerance | 1.2, 0.7, 0.06, 2 deg | PLACEHOLDER | Aim on the field (5e) |
| Teleop | STICK_DEADBAND, TRANSLATION/YAW_EXPO, MAX_* | 0.05, 1.6/1.8, 1.0/0.85 | PLACEHOLDER | Driver preference |
| Auto | SPINUP_TIMEOUT_S, FEED_S_PER_PIECE, FEED_EXTRA_S | 2.0, 0.4, 0.2 | PLACEHOLDER | Time a real volley |
| Auto | State machine speeds (DEFAULT ... OFF_LANE) | 0.8 .. 0.2 | SET BY HAND | Run the auto; slow down where it misses |
| Auto | FLOWER_WAIT_S, GARDEN_WAIT_S, SWEEP_TIMEOUT_S | 1.0, 0.8, 3.0 | SET BY HAND | Run the auto |
| Sim | ShotModel exit angle and grip (test side) | 70 deg, 0.8 | WHAT-IF | Film a shot (5d) |
| Sim | X/Y_POD_OFFSET_IN (BioHorsSim) | 0, 0 | copy | Copy from the robot |

# 9 Attached files and logs

What is in the zip and how to use each part

The zip **BioHORS_team_packet.zip** that comes with this PDF holds everything needed to see what the sim saw, without building anything. The code itself is on GitHub (Section 2).

| Folder in the zip | What | Use it to |
|---|---|---|
| BioHORS_Team_Packet.pdf | This packet | Read |
| logs/*.rlog | Every sim run: two-tip-red / blue (Zenith auto), two-tip-red-physics (real ball physics), state-machine-red / blue, physics-tip-demo(-blue), mechanisms-demo, teleop-aim, ... | Drag onto AdvantageScope (Section 2, step 5) |
| logs/*.trace.json | Per-step timing traces of the Zenith auto | npx zenith sim / calibrate |
| videos/*.mp4 | Screen recordings of the auto in AdvantageScope | Show the team |
| autos/ | robot.json, waypoints.json, two-tip.auto.json, the field file, zenith.json | Open in the Zenith app; the source of the route |
| advantagescope/ | Robot_BioHORS model + config, the hive asset tools | Install the 3D assets |
| docs/ | TUNING.md (the long tuning brief), ROBOT_CAD.md, two-tip.png | Detail behind Sections 4-5 |
| open-values.txt | Every placeholder line in the code, with file and line number | The checklist |
| screenshots/ | The images in this PDF at full size | Slides, posters |

> **NOTE** The logs are regenerated every time the tests run. To make a fresh one after a change, run the tests and look in `TeamCode/build/sim/`.

## Which test writes which log

| Log | Test | What it proves |
|---|---|---|
| two-tip-red / blue | ZenithAutoMatchTest | 12 scored, 2 tips, no contact, under 30 s, both alliances |
| two-tip-red-physics | ZenithAutoMatchTest (physics) | The same with real ball flight and landing |
| state-machine-red / blue | StateMachineAutoMatchTest | The hand-written auto does the same |
| physics-tip-demo | PhysicsTipDemoTest | The hive tips when the balls land, not before |
| mechanisms-demo | MechanismsDemoTest | Intake spins, flower bar moves |
| teleop-aim | MainHorsTeleOpSimTest | Teleop aim turns the chassis onto the hive |

# 10 Links and documentation

Every page this packet points to

| What | Link |
|---|---|
| **Our repository** | https://github.com/ZOGWORK/BioHORS |
| Tuning brief (long version of Sections 3-5) | https://github.com/ZOGWORK/BioHORS/blob/main/docs/TUNING.md |
| Robot CAD notes | https://github.com/ZOGWORK/BioHORS/blob/main/docs/ROBOT_CAD.md |
| Agent / sim rules for this repo | https://github.com/ZOGWORK/BioHORS/blob/main/AGENTS.md |
| **Pedro Pathing docs** | https://pedropathing.com/docs |
| What is new in Pedro 3 | https://pedropathing.com/docs/pathing/pedro3 |
| Tuning overview | https://pedropathing.com/docs/pathing/tuning |
| Mecanum tuning | https://pedropathing.com/docs/pathing/tuning/drivetrain/mecanum |
| Localization | https://pedropathing.com/docs/pathing/tuning/localization |
| Pinpoint | https://pedropathing.com/docs/pathing/tuning/localization/pinpoint |
| Foresight | https://pedropathing.com/docs/pathing/tuning/foresight |
| Constants | https://pedropathing.com/docs/pathing/tuning/constants |
| Tests | https://pedropathing.com/docs/pathing/tuning/test |
| AutoTune (only while on the robot's Wi-Fi) | http://192.168.43.1:10158 |
| Pedro path visualizer | https://visualizer.pedropathing.com/ |
| Pedro Javadoc | https://javadoc.io/doc/com.pedropathing |
| Pedro GitHub / Discord | github.com/Pedro-Pathing / discord.gg/bqRdAjEmjk |
| **Zenith** (auto planner) | https://zenith.horizon36596.org/ |
| **SimLoop** (the simulator) | https://libraries.horizon36596.org/simloop/ |
| SimLoop: what it does not do | https://libraries.horizon36596.org/simloop/limits/ |
| SimLoop: troubleshooting | https://libraries.horizon36596.org/simloop/troubleshooting/ |
| AdvantageScope | https://docs.advantagescope.org/ |
| FTC SDK docs | https://ftc-docs.firstinspires.org/ |
| goBILDA Pinpoint | gobilda.com: Pinpoint odometry computer |

> **TIP** Stuck? In this order: the Pedro Discord for drive tuning; `docs/TUNING.md` for our mechanisms; the SimLoop limits page before blaming the sim.